

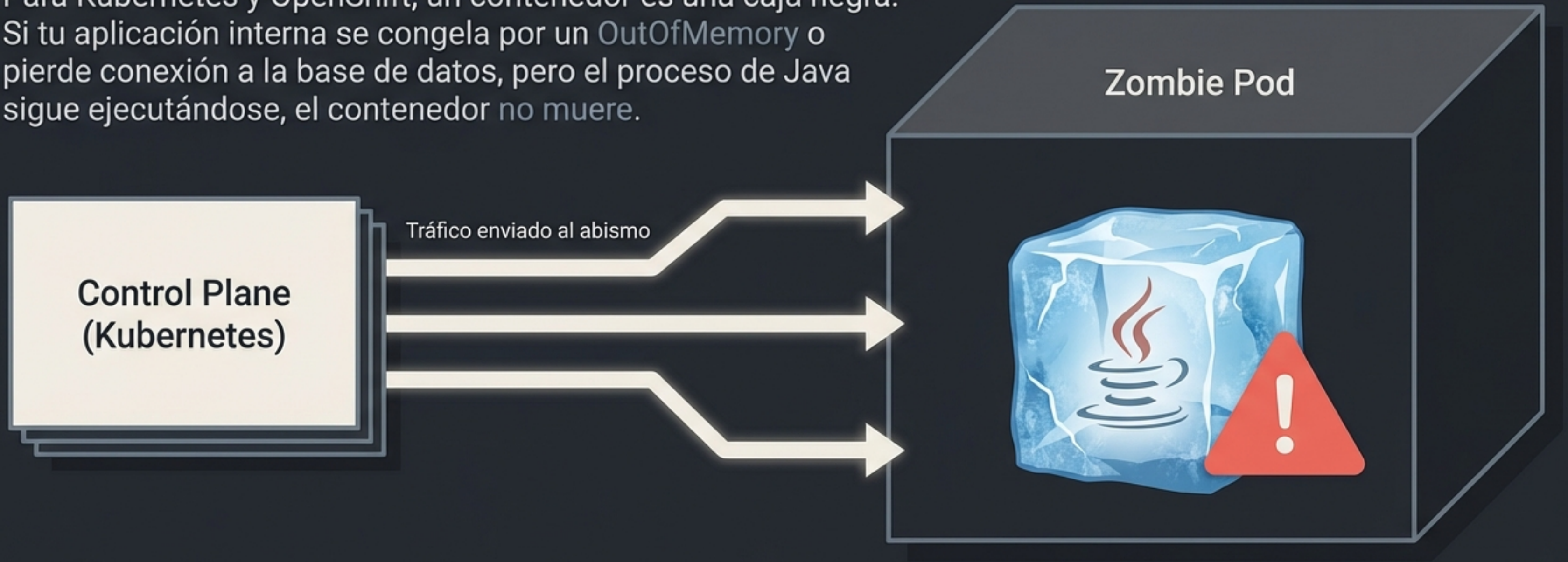
Telemetría Cloud-Native con Spring Boot 4

Guía de supervivencia para la orquestación de microservicios.



El punto ciego del orquestador en la nube

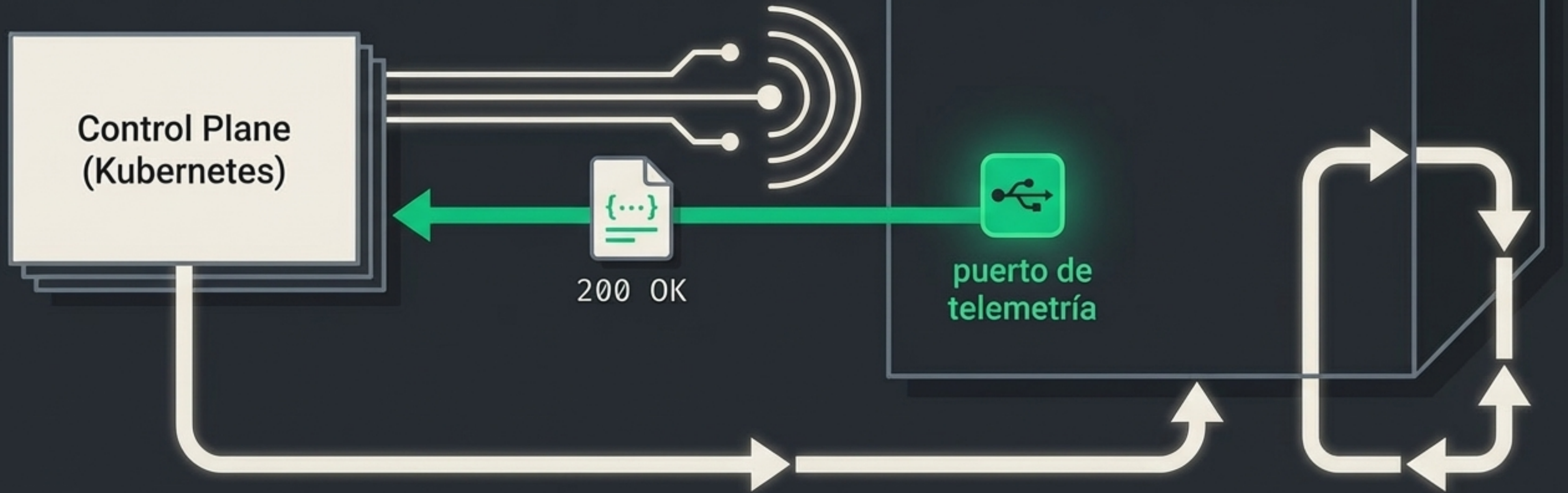
Para Kubernetes y OpenShift, un contenedor es una caja negra. Si tu aplicación interna se congela por un `OutOfMemory` o pierde conexión a la base de datos, pero el proceso de Java sigue ejecutándose, el contenedor no muere.



Un contenedor en ejecución no es sinónimo de una aplicación funcional.

Health Checks: El sistema de telemetría de tu aplicación

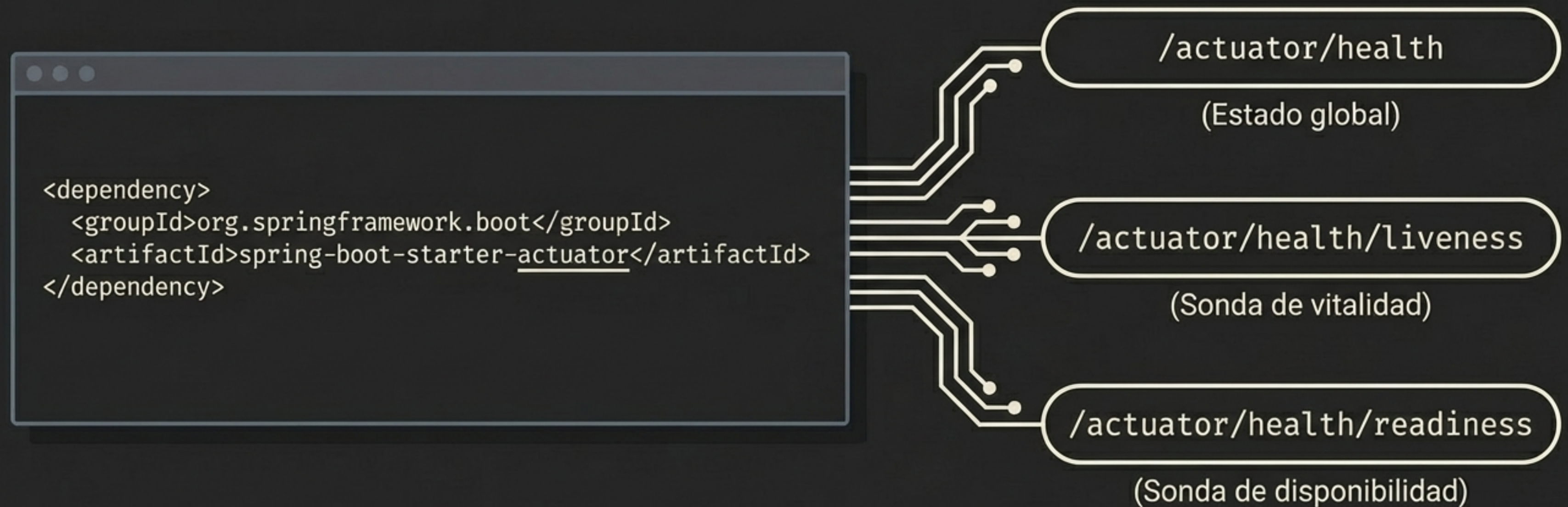
Un health check es una condición expuesta que verifica el estado interno de una aplicación. Actúa como los signos vitales que le dicen a la infraestructura si la aplicación está funcionando como se espera, sin errores críticos o fatales.



Las dos dimensiones de la supervivencia

	Liveness	Readiness
1. ¿Qué pregunta responde?	¿Estoy vivo y sano?	¿Estoy listo para recibir tráfico de usuarios?
2. Condición de éxito	Funciona sin errores fatales internos.	Las dependencias externas (ej. Base de datos, Caché) están disponibles.
3. Ejemplo de fallo	Bloqueo por falta de memoria (OutOfMemory).	La conexión a la base de datos se ha caído.
4. Reacción del Orquestador	Destruye y reinicia el Pod.	Deja de enviarle tráfico temporalmente (lo retira del balanceador).

Anatomía de una dependencia: Spring Boot Actuator



Añadir este starter autoconfigura instantáneamente los endpoints estándar de la industria que Kubernetes espera consumir. No hay que reinventar la rueda.

El lenguaje de la salud: Entendiendo el JSON

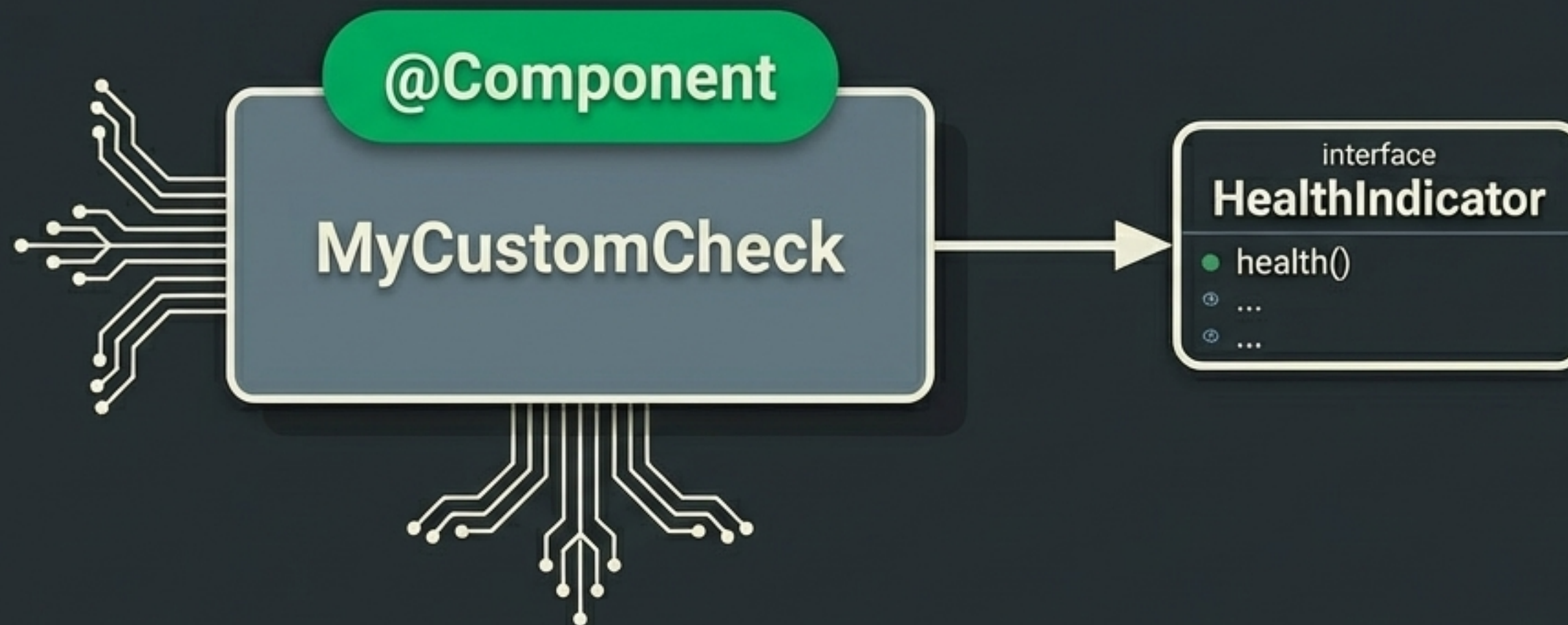
```
{  
  "status": "UP",  
  "components": {  
    "db": { "status": "UP",  
      "details": {  
        "database": "PostgreSQL"  
      } },  
    "diskSpace": { "status": "UP" },  
    "ping": { "status": "UP" }  
  }  
}
```

Estado agregado. Si un componente crítico falla, todo cambia a DOWN.

Actuator detecta automáticamente dependencias como bases de datos en el classpath.

Implementación a medida: La interfaz HealthIndicator

Cuando las comprobaciones automáticas no son suficientes (ej. verificar una API de terceros o una lógica de negocio específica), Spring nos permite inyectar nuestras propias reglas al dashboard principal.



1 Implementar HealthIndicator.

2 Anotar con **@Component** para que el contenedor IoC de Spring lo registre.

3 Retornar el estado usando el patrón Builder.

Construyendo la respuesta con el patrón Builder

```
@Component
public class ExternalApiHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        boolean isApiUp = checkExternalApi(); // Lógica propia
        if (isApiUp) {
            return Health.up()
                .withDetail("API", "Gateway conectado")
                .withDetail("Latency", "45ms")
                .build();
        }
        return Health.down()
            .withDetail("API", "Timeout de conexión").build();
    }
}
```


La línea de tiempo del Orquestador

Mapeando la configuración de OpenShift/Kubernetes a la vida real del contenedor.



Conectando Kubernetes con Spring Boot

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 20
  periodSeconds: 45
```

/actuator/health/liveness

/actuator/health/readiness

Observa cómo `initialDelaySeconds` y `periodSeconds` se definen estrictamente en la infraestructura para gestionar la frecuencia de los latidos generados por Spring Boot.

La coreografía Cloud-Native



Aplicaciones resilientes no son las que nunca fallan, son las que saben comunicar su estado a la infraestructura.